JAVASCRIPT QUICK GUIDE

지도 코드에 사용된 JavaScript 문법

기본 문법 · 카카오 길찾기 · 마커 · GPS 실시간 추적

화살표 함수 · 배열 메서드 · async/await
안전한 객체 접근 · 객체 복사 · 조건 표현

한눈에 보는 문법

문법	한 줄 의미	지도 코드 활용
=>	짧은 함수 표현	응답·배열 요소 검사
!	참과 거짓 반전	API 요청 실패 확인
some()	하나라도 조건 만족	실패 응답·필터 결과 검사
map()	각 요소를 변환	JSON Promise·경로 좌표 변환
forEach()	각 요소마다 실행	마커 제거·경계·경로 처리
async/await	비동기 결과를 순서대로 처리	API·GeoJSON·CSV 로딩
Promise.all()	여러 작업을 동시에 대기	경계 파일 병렬 로딩
?.	객체가 있을 때만 접근	지도·마커 초기화 전 오류 방지
??	null/undefined일 때 기본값	좌표 대체 속성 선택
{ ...obj }	기존 객체 속성 복사	음식점에 거리 속성 추가
조건 ? A : B	조건에 따라 값 선택	마커 표시·숨김

읽는 방법
각 장은 '문법의 뜻 → 코드 예시 → 일반 문법으로 바꾼 모습 → 프로젝트에서 필요한 이유' 순서로 구성했다.

1. 화살표 함수 =>

화살표 함수는 함수를 짧게 작성하는 문법이다. 매개변수는 화살표 왼쪽, 실행 결과는 오른쪽에 작성한다.

> ★ Q&A; 예상 질문
> 화살표 함수를 일반 function 문법으로 바꾸면 어떻게 되나요?

화살표 함수

```
response => !response.ok
```

일반 함수로 바꾸면

```
function (response) {
  return !response.ok;
}
```

중괄호를 생략하면 오른쪽 표현식의 결과가 자동 반환된다. 이 예시는 response를 입력받아 요청 실패 여부를 true 또는 false로 반환한다.

2. 부정 연산자 !

!는 Boolean 값을 반대로 바꾼다. true는 false가 되고 false는 true가 된다.

기본 예시

```
!true;  // false
!false; // true
```

API 응답 검사

```
response.ok   // 성공이면 true
!response.ok  // 실패이면 true
```

response.ok는 HTTP 상태 코드가 200~299이면 true다. 앞에 !를 붙이면 '응답이 성공하지 않았는가?'라는 실패 검사로 바뀐다.

3. some()

some()은 배열 요소 중 하나라도 조건을 만족하면 true를 반환한다.

> ★ Q&A; 예상 질문
> some()은 언제 반복을 중단하고 어떤 값을 반환하나요?

사용 예시

```
const hasFailure = responses.some(
  response => !response.ok
);
```

응답을 하나씩 검사하다가 실패 응답을 찾으면 즉시 true를 반환하고 반복을 끝낸다. 실패가 하나도 없으면 false다.

> 세 문법을 합치면
> responses.some(response => !response.ok)는 '응답 배열 중 실패한 응답이 하나라도 있는가?'라는 뜻이다.

4. map()

map()은 기존 배열의 각 요소를 변환해 같은 길이의 새로운 배열을 만든다.

응답을 JSON 변환 작업으로 바꾸기

```
const jsonPromises = responses.map(
  response => response.json()
);
```

responses 배열 자체는 바뀌지 않는다. 각 Response를 response.json() Promise로 변환한 새 배열이 jsonPromises에 저장된다.

5. forEach()

forEach()는 배열의 각 요소에 대해 같은 코드를 실행한다.

음식점마다 마커 생성

```
restaurants.forEach(restaurant => {
  createMarker(restaurant);
});
```

forEach()의 주목적은 반복 실행이다. map()처럼 변환 결과를 모아 새로운 배열로 반환하지 않는다.

map()과 forEach() 비교

구분	map()	forEach()
목적	요소 변환	요소마다 작업 실행
반환	새 배열	보통 사용하지 않음
지도 예시	경로 좌표를 LatLng로 변환	마커를 하나씩 제거

6. async와 await

async/await는 API 요청이나 파일 로딩처럼 바로 끝나지 않는 비동기 작업을 위에서 아래 순서로 읽기 쉽게 처리한다.

> ★ Q&A; 예상 질문
> await가 브라우저 전체 실행을 멈추는 것인가요?

사용 예시

```
async function loadData() {
  const response = await fetch(DATA_URL);
  const data = await response.json();
  return data;
}
```

async는 함수 안에서 await를 사용할 수 있게 하고 함수 반환값을 Promise로 만든다. await는 Promise 작업이 끝나 실제 결과가 준비될 때까지 해당 async 함수의 다음 코드 실행을 기다린다.

왜 둘을 함께 사용하는가?

API 응답이 오기 전에 response.ok를 확인하거나, JSON 변환이 끝나기 전에 경로를 그리면 올바른 결과를 사용할 수 없다. async/await는 필요한 결과가 준비된 다음 단계가 실행되도록 순서를 보장한다.

> 중요
> await는 브라우저 전체를 멈추지 않는다. 현재 async 함수의 다음 실행만 기다리며, 화면 표시와 다른 이벤트는 계속 처리된다.

7. Promise.all()

Promise.all()은 여러 비동기 작업을 동시에 시작하고 모든 작업이 끝날 때까지 기다린다.

> ★ Q&A; 예상 질문
> Promise.all()을 순차 await 대신 사용한 이유는 무엇인가요?

두 GeoJSON 파일 병렬 로딩

```
const responses = await Promise.all([
  fetch(DISTRICT_URL),
  fetch(CITY_URL)
]);
```

첫 파일이 끝난 뒤 두 번째 파일을 요청하는 순차 방식보다 효율적이다. 단, 포함된 작업 중 하나라도 reject되면 Promise.all() 전체가 실패한다.

8. 선택적 연결 연산자 ?.

?.는 왼쪽 객체가 null 또는 undefined가 아닐 때만 다음 속성이나 함수를 사용한다.

> ★ Q&A; 예상 질문
> routePolyline?.setMap(null)에서 ?.가 필요한 이유는 무엇인가요?

짧은 코드

```
infoWindow?.close();
```

if문으로 바꾸면

```
if (infoWindow !== null &&
    infoWindow !== undefined) {
    infoWindow.close();
}
```

지도나 정보창이 아직 생성되지 않은 첫 실행에서도 오류가 나지 않게 한다. routePolyline?.setMap(null)도 이전 경로선이 있을 때만 제거한다는 뜻이다.

9. Null 병합 연산자 ??

??는 왼쪽 값이 null 또는 undefined일 때만 오른쪽 기본값을 사용한다.

> ★ Q&A; 예상 질문
> 기본값을 지정할 때 ??와 ||의 차이는 무엇인가요?

좌표 속성 대체

```
const value =
    target?.latitude ?? target?.lat;
```

latitude 속성이 없으면 lat을 대신 사용한다. 숫자 0과 빈 문자열은 유효한 값으로 유지한다는 점에서 ||와 다르다.

??와 || 비교

```
0 || 10;  // 10: 0을 falsy로 판단
0 ?? 10;  // 0 : null/undefined가 아니므로 유지

'' || '기본값'; // '기본값'
'' ?? '기본값'; // '' 유지
```

10. 구조 분해 할당

구조 분해 할당은 객체나 배열에서 필요한 값을 바로 꺼내 변수로 사용한다.

마커 객체에서 필요한 속성 꺼내기

```
restaurantMarkers.forEach(
  ({ restaurants, marker }) => {
    marker.setMap(
      restaurants.some(matchesFilter)
        ? map
        : null
    );
  }
);
```

restaurantMarkers의 각 요소에서 restaurants와 marker 속성을 바로 꺼낸다. item.restaurants, item.marker를 반복해서 작성하지 않아도 된다.

11. 전개 문법 ...

객체 앞의 ...은 기존 객체의 속성을 새 객체에 복사한다.

음식점 정보에 거리 추가

```
return {
  ...restaurant,
  distanceKm,
  distanceLabel
};
```

이름, 주소, 좌표 등 기존 restaurant 속성은 그대로 유지하고 distanceKm와 distanceLabel을 추가한 새 객체를 만든다.

12. 삼항 연산자

삼항 연산자는 조건에 따라 두 값 중 하나를 선택하는 짧은 조건문이다.

> ★ Q&A; 예상 질문
> visible ? map : null을 if문으로 풀어쓰면 어떻게 되나요?

삼항 연산자

```
marker.setMap(visible ? map : null);
```

if문으로 바꾸면

```
if (visible) {
  marker.setMap(map);
} else {
  marker.setMap(null);
}
```

visible이 true면 마커를 map에 연결해 표시하고, false면 null에 연결해 지도에서 숨긴다. 두 코드는 같은 의미다.

13. 프로젝트 모듈: IIFE와 window 공개

길찾기 서비스는 내부 코드를 즉시 실행 함수(IIFE)로 감싸고, 다른 파일에 필요한 함수만 window에 공개한다.

```
// 내부 변수와 함수를 별도 영역에 감싼다.
(function () {
  const DIRECTIONS_ENDPOINT = 'https://apis-navi.kakaomobility.com/...';

  // 자동차 길찾기 요청을 담당하는 비동기 함수다.
  async function findDrivingRoute(origin, destination) {
    // 길찾기 요청 처리
  }

  // 다른 JavaScript 파일에서 사용할 함수만 공개한다.
  window.DirectionsService = { findDrivingRoute };
})();
```

(function () { ... })();는 함수를 정의하자마자 실행한다. 내부 변수는 전역 영역과 분리된다. { findDrivingRoute }는 { findDrivingRoute: findDrivingRoute }의 단축형이다.

> ★ Q&A; 예상 질문
> 왜 모든 함수를 전역에 만들지 않고 IIFE로 감싼 뒤 필요한 기능만 window에 공개하나요?

14. 카카오 지도 SDK 동적 로딩

```
// 저장된 JavaScript 키가 있으면 앞뒤 공백을 제거한다.
const apiKey = window.KAKAO_JS_MAPS_API_KEY?.trim();

// 키가 없거나 32자리 형식이 아니면 실행을 중단한다.
if (!apiKey || !/^[0-9a-f]{32}$/i.test(apiKey)) {
  throw new Error('카카오 JavaScript 키가 올바르지 않습니다.');
}

// 카카오 지도 SDK를 불러올 script 태그를 만든다.
const script = document.createElement('script');
script.src =
  `https://dapi.kakao.com/v2/maps/sdk.js?appkey=${encodeURIComponent(apiKey)}&autoload=false`;
script.async = true; // 다른 화면 처리를 막지 않고 내려받는다.
script.onload = initializeKakaoMap; // 로딩 성공 후 지도를 만든다.
script.onerror = () => showMapError('지도 SDK 로딩 실패'); // 실패 처리
document.head.appendChild(script); // 문서에 붙여 실제 로딩을 시작한다.
```

?.trim()은 키가 있을 때만 공백을 제거한다. 정규식 test()는 키 형식을 검사한다. 템플릿 리터럴의 ${...}는 문자열에 값을 넣으며 encodeURIComponent()는 URL 값을 안전하게 변환한다. script는 appendChild()로 문서에 붙여야 실제 로딩이 시작된다.

> ★ Q&A; 예상 질문
> script.async, onload, onerror와 appendChild()는 각각 왜 필요한가요?

15. 카카오 자동차 길찾기 문법

요청 주소와 인증 헤더

```
// 출발지와 도착지를 URL 검색 조건으로 만든다.
const query = new URLSearchParams({
  // 카카오 길찾기 API는 '경도,위도' 순서를 사용한다.
  origin: `${origin.longitude},${origin.latitude}`,
  destination: `${destination.longitude},${destination.latitude}`
});

// 완성된 주소로 카카오 길찾기 REST API를 호출한다.
const response = await fetch(
  `${DIRECTIONS_ENDPOINT}?${query}`,
  // REST API 키를 인증 헤더에 넣는다.
  { headers: { Authorization: `KakaoAK ${getApiKey()}` } }
);
```

URLSearchParams가 쿼리 문자열을 만든다. 카카오 길찾기는 경도,위도 순서로 좌표를 받는다. Authorization은 REST API 키를 KakaoAK 뒤에 붙여 보내는 인증 헤더다.

JSON에서 경로 좌표 꺼내기

```
// HTTP 요청이 실패했다면 경로 처리를 중단한다.
if (!response.ok) throw new Error('길찾기 요청 실패');

// 응답 본문을 JavaScript 객체로 변환한다.
const data = await response.json();
const route = data.routes?.[0]; // 첫 번째 추천 경로를 선택한다.

// 경로 구간과 도로를 차례대로 순회한다.
route.sections?.forEach(section => {
  section.roads?.forEach(road => {
    // 좌표가 경도와 위도의 쌍이므로 두 칸씩 이동한다.
    for (let index = 0; index < road.vertexes.length; index += 2) {
      points.push({
        longitude: road.vertexes[index],
        latitude: road.vertexes[index + 1]
      });
    }
  });
});
```

response.ok가 false면 throw로 실행을 catch로 보낸다. routes?.[0]은 첫 경로가 있을 때만 꺼낸다. vertexes는 [경도, 위도, 경도, 위도 ...] 구조라 index += 2로 두 칸씩 이동한다.

> ★ Q&A; 예상 질문
> 왜 vertexes 반복문은 index++가 아니라 index += 2를 사용하나요?

좌표를 카카오 경로선으로 그리기

```
// 이전 검색에서 그린 경로선이 있으면 제거한다.
routePolyline?.setMap(null);

// 일반 좌표를 카카오 지도 좌표 객체로 바꾼다.
const path = route.points.map(point =>
  new kakao.maps.LatLng(point.latitude, point.longitude)
);

// 변환한 좌표들을 연결해 지도에 경로선을 그린다.
routePolyline = new kakao.maps.Polyline({ map, path });

// 경로 전체가 보이도록 지도 범위를 계산한다.
const bounds = new kakao.maps.LatLngBounds();
path.forEach(position => bounds.extend(position));
map.setBounds(bounds);
```

이전 경로선을 제거하고, 일반 좌표를 카카오 LatLng 객체로 변환해 Polyline을 만든다. LatLngBounds와 setBounds()는 전체 경로가 화면에 들어오도록 지도 범위를 맞춘다.

> ★ Q&A; 예상 질문
> API 좌표를 왜 바로 쓰지 않고 kakao.maps.LatLng 객체로 변환하나요?

16. 길찾기 UI의 try / catch / finally

```
// 중복 요청을 막기 위해 길찾기 버튼을 잠근다.
routeButton.disabled = true;
try {
  // API의 경로 결과가 올 때까지 기다린다.
  const route = await DirectionsService.findDrivingRoute(origin, destination);
  drawRoute(route); // 받은 경로를 지도에 그린다.
} catch (error) {
  showRouteError(error.message); // 실패 이유를 화면에 표시한다.
} finally {
  routeButton.disabled = false; // 성공·실패와 관계없이 잠금을 푼다.
}
```

try는 요청을 시도하고 catch는 오류를 처리한다. finally는 성공·실패와 상관없이 실행되므로 요청 중 비활성화한 버튼을 반드시 되돌린다.

17. 카카오 마커 생성 · 초기화 · 묶기

기존 마커 제거

```
// 기존 마커를 하나씩 지도에서 제거한다.
restaurantMarkers.forEach(({ marker }) =>
  marker.setMap(null)
);
restaurantMarkers = []; // 마커 목록도 빈 배열로 초기화한다.
```

구조 분해로 marker만 꺼내 setMap(null)로 지도 연결을 끊는다. 카카오 지도도 마커 객체가 필요하며, 다시 그리기 전에 제거하지 않으면 이전 마커와 새 마커가 겹친다.

같은 좌표의 음식점 묶기

```
// 같은 좌표의 음식점들을 묶어 둘 저장소다.
const groups = new Map();

restaurants.forEach(restaurant => {
  // 위도와 경도를 하나의 그룹 키로 만든다.
  const key = `${restaurant.latitude.toFixed(8)},${restaurant.longitude.toFixed(8)}`;
  const group = groups.get(key) || []; // 기존 그룹 또는 빈 배열
  group.push(restaurant); // 현재 음식점을 그룹에 추가한다.
  groups.set(key, group); // 변경된 그룹을 다시 저장한다.
});
```

new Map()은 키와 값을 저장한다. 같은 좌표 문자열을 key로 사용해 한 건물의 여러 음식점을 한 그룹으로 묶는다. groups.get(key) || []는 기존 그룹이 없을 때 빈 배열을 사용한다. toFixed(8)은 미세한 소수점 차이로 그룹이 갈라지는 것을 줄인다.

마커 만들기와 클릭 이벤트

```
// 음식점 좌표에 표시할 카카오 마커를 만든다.
const marker = new kakao.maps.Marker({
  map,
  position: new kakao.maps.LatLng(latitude, longitude)
});

// 사용자가 마커를 클릭하면 음식점 정보를 연다.
kakao.maps.event.addListener(marker, 'click', () => {
  openRestaurantInfo(restaurants);
});
```

new는 카카오 Marker 인스턴스를 만든다. addListener는 마커 클릭 때 화살표 함수가 실행되도록 콜백을 등록한다.

> ★ Q&A; 예상 질문
> 마커를 다시 표시할 때 기존 마커에 setMap(null)을 먼저 호출하는 이유는 무엇인가요?

18. GPS 현재 위치와 실시간 추적

GPS 정확도 요청 설정

```
const DEFAULT_OPTIONS = {
  // 가능한 경우 GPS 등 정확도가 높은 위치 측정 방법을 요청한다.
  enableHighAccuracy: true,

  // 위치 응답을 최대 10초까지 기다린다.
  timeout: 10000,

  // 저장된 과거 위치 대신 새 위치를 요청한다.
  maximumAge: 0
};
```

enableHighAccuracy는 브라우저에 더 정확한 측정을 요청하지만 정확한 좌표를 반드시 보장하지는 않는다. timeout은 대기 제한 시간이고 maximumAge는 캐시된 위치를 허용할 시간이다. 현재 프로젝트는 maximumAge를 0으로 지정해 가능한 한 새 위치를 요청한다.

> 중요한 구분
> 이 설정은 GPS 좌표를 수학적으로 고치는 '좌표 보정 알고리즘'이 아니다. 브라우저가 제공하는 위치 측정 방식과 캐시 사용 조건을 설정하는 코드다.

현재 위치 1회 가져오기: Promise 변환

```
// 콜백 방식의 GPS API를 await 가능한 Promise로 감싼다.
function getCurrentPosition(options = {}) {
  return new Promise((resolve, reject) => {
    navigator.geolocation.getCurrentPosition(
      position => resolve({
        latitude: position.coords.latitude,   // 현재 위도
        longitude: position.coords.longitude, // 현재 경도
        accuracy: position.coords.accuracy    // 예상 오차 반경(m)
      }),
      reject,  // 오류가 나면 Promise를 실패 처리한다.
      // 기본 설정에 전달받은 설정을 덮어쓴다.
      { ...DEFAULT_OPTIONS, ...options }
    );
  });
}
```

브라우저 GPS API는 성공·실패 콜백 방식이다. new Promise로 감싸면 호출하는 쪽에서 await를 사용할 수 있다. resolve는 위치 성공값, reject는 오류를 전달한다. 전개 문법은 기본 옵션 뒤에 사용자 옵션을 덮어쓴다.

> ★ Q&A; 예상 질문
> accuracy 값은 좌표를 보정한 결과인가요, 아니면 현재 위치의 예상 오차 범위인가요?

위치가 바뀔 때마다 받기: watchPosition

```
// 실행 중인 위치 추적 번호를 보관한다.
let watchId = null;

function startWatching(onPosition, onError) {
  // 위치가 바뀔 때마다 onPosition을 실행한다.
  watchId = navigator.geolocation.watchPosition(
    onPosition,
    onError,
    DEFAULT_OPTIONS
  );
  return watchId;
}

function stopWatching() {
  if (watchId !== null) {
    // 저장해 둔 번호로 실시간 GPS 추적을 중단한다.
    navigator.geolocation.clearWatch(watchId);
    watchId = null;
  }
}
```

watchPosition은 위치가 갱신될 때마다 onPosition 콜백을 실행하고 추적 번호를 반환한다. watchId는 클로저에 보관된다. clearWatch는 GPS 추적을 중단해 불필요한 배터리와 자원 사용을 막는다. 페이지 종료 시에도 중단하는 것이 안전하다.

> ★ Q&A; 예상 질문
> getCurrentPosition과 watchPosition의 차이, watchId를 저장해야 하는 이유는 무엇인가요?

19. 현재 위치 마커를 재사용하는 조건문

```
// 현재 위치 표시가 이미 있으면 좌표만 갱신한다.
if (currentLocationMarker) {
  currentLocationMarker.setPosition(position);
  currentLocationMarker.setMap(map);
} else {
  // 최초 실행일 때만 현재 위치 표시를 새로 만든다.
  currentLocationMarker = new kakao.maps.CustomOverlay({
    map,
    position,
    content: createCurrentLocationElement()
  });
}

// 최초 한 번만 지도를 현재 위치로 이동한다.
if (!centeredOnLocation) {
  map.panTo(position);
  centeredOnLocation = true;
}
```

이미 현재 위치 표시가 있으면 새로 만들지 않고 좌표만 갱신한다. 처음에만 CustomOverlay를 만든다. centeredOnLocation 플래그는 최초 한 번만 지도를 현재 위치로 이동시켜, 실시간 GPS 갱신 때마다 사용자가 보던 화면이 강제로 움직이는 것을 막는다.

> ★ Q&A; 예상 질문
> GPS가 갱신될 때마다 새 마커를 만들지 않고 기존 위치 표시를 재사용하는 이유는 무엇인가요?

20. 주변 맛집 계산: flatMap · sort · slice

```
// 모든 구역을 검사하고 결과를 하나의 배열로 합친다.
const nearby = districts.flatMap(district => {
  // 하버사인 공식으로 현재 위치와 구역 중심의 거리를 구한다.
  const distanceKm = haversineDistance(current, district.center);
  if (distanceKm > radiusKm) return []; // 검색 반경 밖이면 제외한다.

  // 음식점 원본 정보에 계산한 거리를 추가한다.
  return district.restaurants.map(restaurant => ({
    ...restaurant,
    distanceKm
  }));
});

// 가까운 음식점이 앞으로 오도록 오름차순 정렬한다.
nearby.sort((a, b) => a.distanceKm - b.distanceKm);
return nearby.slice(0, limit); // 필요한 개수만 잘라 반환한다.
```

flatMap은 각 결과를 하나의 배열로 평탄화한다. 범위 밖이면 []를 반환해 제외한다. sort의 비교 함수가 음수면 a가 앞으로 가므로 가까운 순서가 된다. slice(0, limit)는 앞에서 필요한 개수만 복사한다. 하버사인 공식은 GPS 위·경도로 두 지점의 직선거리를 구해 주변 맛집을 판별하기 위해 사용한다.

> ★ Q&A; 예상 질문
> flatMap에서 조건에 맞지 않을 때 null이 아니라 빈 배열 []을 반환하는 이유는 무엇인가요?

21. CustomEvent로 지도 파일과 화면 파일 연결

```
// 지도 생성 완료 사실과 지도 객체를 다른 파일에 알린다.
window.dispatchEvent(new CustomEvent('restaurant-map-ready', {
  detail: { map }
}));

// 지도 준비 이벤트가 발생할 때까지 기다린다.
window.addEventListener('restaurant-map-ready', event => {
  const map = event.detail.map; // detail에서 지도 객체를 꺼낸다.
  initializeRouteController(map); // 길찾기 기능을 초기화한다.
});
```

CustomEvent는 한 파일에서 발생한 상태 변화를 다른 파일에 알린다. detail에 지도 객체를 담아 전달하며, 받는 쪽은 addEventListener로 기다린다. 파일끼리 서로의 내부 함수를 강하게 의존하지 않고 초기화 시점을 맞출 수 있다.

> ★ Q&A; 예상 질문
> 지도 준비 여부를 전역 변수만 반복 확인하지 않고 CustomEvent를 사용한 이유는 무엇인가요?

22. 프로젝트 특수 문법 빠른 표

기능	주요 문법·API	프로젝트에서 하는 일
지도 SDK	createElement, onload	카카오 지도 스크립트 로딩
자동차 길찾기	fetch, URLSearchParams	REST API에 출발·도착 좌표 전송
경로 표시	LatLng, Polyline, Bounds	좌표 변환·선 그리기·화면 맞춤
맛집 마커	Marker, Map, addListener	마커 생성·좌표 그룹·클릭 처리
현재 위치	getCurrentPosition	GPS 위치를 한 번 조회
실시간 위치	watchPosition, clearWatch	위치 변화 추적·중지
주변 맛집	flatMap, sort, 하버사인	거리 계산·필터·가까운 순 정렬
파일 연결	CustomEvent, detail	지도 준비 상태와 객체 전달

마지막 정리

지도 코드의 JavaScript 문법은 서로 따로 동작하기보다 조합되어 사용된다. 아래 문장을 해석할 수 있으면 핵심 문법을 이해한 것이다.

```
if (responses.some(response => !response.ok)) {
  throw new Error('파일을 불러오지 못했습니다.');
}
```

해석: responses 배열을 하나씩 확인하고, ok가 false인 응답이 하나라도 있으면 오류를 발생시킨다.

```
restaurantMarkers.forEach(({ marker }) =>
  marker.setMap(null)
);
```

해석: 마커 배열의 각 요소에서 marker를 꺼내 지도 연결을 해제한다.

```
const response = await fetch(API_URL);
const data = await response.json();
```

해석: API 응답을 기다려 실제 Response를 받고, 응답 본문을 JSON 객체로 변환할 때까지 다시 기다린다.

질문	핵심 답변
async를 왜 쓰나?	await 사용과 Promise 반환을 위해
await를 왜 쓰나?	완료 결과를 받은 뒤 다음 코드를 실행하기 위해
some()을 왜 쓰나?	하나라도 조건을 만족하는지 확인하기 위해
?.를 왜 쓰나?	객체 생성 전 null/undefined 오류를 막기 위해
map()과 forEach() 차이?	새 배열 변환과 단순 반복 실행의 차이

원본: txt/map_code/06_자바스크립트_문법설명.txt